

Test Plan

The Job Journal

Version 1.0
May 14, 2025

The Workforce Center of Blackbriar
Blackbriar City
Amber Kansfield

Contents

Section 1. Overview	3
1.1. Purpose	3
1.2. Audience	3
1.3. Product Analysis	3
Section 2. Test Plan	3
2.2. Scope	3
2.3. Objectives.....	3
2.4. Procedure	4
2.4.a JUnit.....	4
2.4.b Manual.....	4
2.5. Resource Requirements	4
2.6. Exit Criteria	4
Section 3. JUnit Tests.....	4
3.2 Test Classes.....	4
3.2.a UserTests.....	5
3.2.b ApplicationTests	7
3.2 JUnit Test Results	8
3.2.a User Tests 1	8
3.2.b User Tests 2	9
3.2.c Application Tests 1	9
Section 4. Manual Test Results	12
4.3.a	12
4.3.b	13
Section 6. Summary.....	13

Section 1. Overview

1.1. Purpose

The purpose of this document is to detail the testing process for *The Job Journal* project. It will cover the overall test plan, showcase the test classes in use, and list the test cases and their results.

1.2. Audience

The intended audience for this document is the customer, the Workforce Center of Blackbriar. Further audiences include any relevant stakeholders and the development team, as testing is carried out.

1.3. Product Analysis

The Job Journal is a web-based application that allows users to keep track of their various job applications. It is meant to be a lightweight and simple organizational tool. Users can sign in and add an application to their list. These applications require a job title and employer and have optional fields for date due, date applied, and date posted.

The user will be able to download a CSV file of their applications, which will save to their PC.

Section 2. Test Plan

2.2. Scope

The JUnit Tests will cover user sign-up and the various applications that can be submitted. Front-end logic is considered out of scope for the server-side JUnit tests. However, as the front-end logic is fairly minimalistic and mostly aesthetically focused, manual testing will be conducted to ensure an optimal user experience.

2.3. Objectives

Objectives are to ensure error-resistant application submission and to identify any user experience flaws that may be present in the application.

2.4. Procedure

2.4.a JUnit

1. Ensure the testing environment mirrors the production environment.
2. Run JUnit tests.
3. Note the result of each test in the test case document, taking note of failed tests in particular.
4. Determine the root cause of the failure.
5. Revise the code or solve other issues causing the problem.
6. Run unit tests again.

2.4.b Manual

1. Check UI functionality.
2. Note whether the action resulted in the expected response.
3. Revise the code that caused the failure.
4. Repeat until functionality is as expected.

2.5. Resource Requirements

1. JUnit
2. Mockito (Pre-included)

2.6. Exit Criteria

JUnit – The ability to create, edit, and delete Applications should be facilitated. Users should be able to be created, and their emails should be unique. Each JUnit Test should have a result of “Pass.”

Manual – Following the directions of the test case should result in the expected behavior as written in the test case.

Section 3. JUnit Tests

3.2 Test Classes

See Screenshots below for the test classes implemented:

3.2.a UserTests

```
public class UserTests {  
  
    @AfterEach new *  
    void deleteData(){  
        List<User> users = userDao.findByFirstName("Test7132");  
        userDao.deleteAll(users);  
    }  
  
    // Utilizes simulated front end input.  
    // Tests that a user can effectively be saved, as well as if the password is properly encrypted.  
    @ParameterizedTest new *  
    @JsonFileSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/duplicateEmail.json")  
    void createUser(Map<String, Object> stringObjectMap) {  
        User user = parserService.parseUser(stringObjectMap);  
        User savedUser = userDao.save(user);  
  
        // Assert that user has an ID and that it has been changed from the default.  
        assert savedUser.getId() != null && savedUser.getId() != -1;  
  
        // Asserts that the password has been encrypted.  
        assert savedUser.getPassword() != stringObjectMap.get("password");  
    }  
  
    // Asserts that a user is findable by a known email  
    @ParameterizedTest new *  
    @CsvSource({"email@email.com"})  
    void findByEmail(String email){  
        User user = userDao.findByEmail(email);  
        assert user != null;  
    }  
};
```

```
// Asserts that a user cannot enter an email that has already been used.  
@ParameterizedTest new *  
@JsonFileSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/duplicateEmail.json")  
void duplicateEmailThrowsException(Map<String, Object> jsonPayload) {  
    User user1 = parserService.parseUser(jsonPayload);  
    User user2 = parserService.parseUser(jsonPayload);  
    userDao.save(user1);  
  
    Exception e = assertThrows(Exception.class, () -> {  
        userDao.save(user2);  
    });  
}
```

3.2.b ApplicationTests

```

public class ApplicationTests {

    // An application with no due date and an applied date should be able to be saved
    @Test new *
    @JsonSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/appliedApplication.json")
    void createCompleteApplication(Map<String, Object> jsonPayload) {
        Application application = parserService.parseApplication(jsonPayload);

        // Asserts that date due is null and that date applied has a value.
        assert application.getDateDue() == null;
        assert application.getDateApplied() != null;

        Application savedApplication = applicationDAO.save(application);
        // Asserts that the application has been saved and that the ID has been changed from default.
        assert savedApplication != null;
        assert savedApplication.getId() != -1;
    }

    // An application with a due date and no applied date should be able to be saved
    @Test new *
    @JsonSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/unfinishedApplication.json")
    void createUnfinishedApplication(Map<String, Object> jsonPayload) {
        Application application = parserService.parseApplication(jsonPayload);

        assert application.getDateDue() != null;
        assert application.getDateApplied() == null;

        Application savedApplication = applicationDAO.save(application);

        // Asserts that the application has been saved and that the ID has been changed from default.
        assert savedApplication != null;
        assert savedApplication.getId() != -1;
    }

    //Applications with no due date or applied date will fail to save
    @Test new *
    @JsonSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/noDateApplication.json")
    void rejectApplicationWithNoDates(Map<String, Object> jsonPayload) {
        Exception e = assertThrows(Exception.class, () -> {
            Application application = parserService.parseApplication(jsonPayload);
        });
    }
}

```

```

// Update Functionality
@Test new *
@JsonSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/appliedApplication.json")
void updateApplication(Map<String, Object> jsonPayload) {
    Application application = parserService.parseApplication(jsonPayload);
    Application savedApplication = applicationDAO.save(application);

    savedApplication.setJobTitle("New");
    savedApplication.setEmployer("New");
    Application editedApplication = applicationDAO.save(savedApplication);
    assertEquals(savedApplication.getEmployer(), editedApplication.getEmployer());
    assertEquals(savedApplication.getJobTitle(), editedApplication.getJobTitle());
}

// Delete Functionality
@Test new *
@JsonSource("src/test/java/edu/wgu/jobjournalcapstone/TestFiles/appliedApplication.json")
void deleteApplication(Map<String, Object> jsonPayload){
    Application application = parserService.parseApplication(jsonPayload);
    Application savedApplication = applicationDAO.save(application);

    Integer id = application.getId();
    assertNotNull(id);
    applicationDAO.delete(savedApplication);
    assertNull(applicationDAO.findById(id.longValue()));
}
}

```

3.2 JUnit Test Results

3.2.a User Tests 1

✖ UserTests (edu.wgu.jobjournalcapstone) ✖ duplicateEmailThrowsException(Map) ✖ [1] jsonPayload={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=password} ✓ findByEmail(String) ✓ createUser(Map) ✓ [1] stringObjectMap={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=password}	790 ms 745 ms 745 ms 15 ms 30 ms 30 ms

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
1	Duplicate Email Throws Exception	(JSON) 2 Users with identical email addresses.	The second user is expected to throw an error.	Both users were saved to the database.	FAIL
2	Find By Email	String of a known email ("email@email.com")	The user will be found and will not return null.	The user was found under the input email.	PASS
3	Create User	(JSON) User with all fields	The user will be saved to the database with an ID	The user was saved as expected.	PASS

3.2.b User Tests 2

✓	✓	UserTests (edu.wgu.jobjournalcapstone)	872 ms
✓	✓	duplicateEmailThrowsException(Map)	825 ms
✓	✓	[1] jsonPayload={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=password}	825 ms
>	✓	findByEmail(String)	17 ms
✓	✓	createUser(Map)	30 ms
✓	✓	[1] stringObjectMap={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=password}	30 ms

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
4	Duplicate Email Throws Exception	(JSON) 2 Users with identical email addresses.	The second user is expected to throw an error.	The second user threw an error as expected.	PASS

3.2.c Application Tests 1

✓	!	ApplicationTests (edu.wgu.jobjournalcapstone)	793 ms
✓	✓	createCompleteApplication(Map)	660 ms
✓	✓	[1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14T03:07:17.143Z}	660 ms
>	!	createUnfinishedApplication(Map)	16 ms
>	✓	rejectApplicationWithNoDates(Map)	23 ms
✓	✗	updateApplication(Map)	58 ms
✓	✗	[1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14T03:07:17.143Z}	58 ms
✓	✗	deleteApplication(Map)	36 ms
✓	✗	[1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14T03:07:17.143Z}	36 ms

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
5	Create Complete Application	(JSON) Application with an applied date and a null due date.	The application should save to the database, and an ID should be generated.	The application was correctly saved to the database.	PASS
6	Create Unfinished Application	(JSON) Application with null applied date and a due date.	The application should save to the database, and an ID should be generated.	The application failed to save to the database.	FAIL
7	Reject Application with no dates	(JSON) Application with both null applied and due dates.	The application should be rejected if both the due date and the applied date are null.	The application was not saved to the database.	PASS
8	Update Application	(JSON) Application with applied date and null due date.	The application information should be updated, and the new data should not match the old data.	The old data and the new data were determined to be equal.	FAIL
9	Delete Application	(JSON) Application with applied date and null due date.	The saved application should be removed from the database, and its ID should not bring up an application.	A null value was expected, but an optional value was received. (Likely an issue with the test syntax.)	FAIL

```

  ✓ ApplicationTests (edu.wgu.jobjournalcapstone) 551 ms
    ✓ createUnfinishedApplication(Map) 551 ms
      ✓ [1] jsonPayload={id=-1, dateCreated=2025-05-14T03:10:37.538Z, dateUpdated=2025-05-14 551 ms

```

```

  ✓ ApplicationTests (edu.wgu.jobjournalcapstone) 544 ms
    ✓ updateApplication(Map) 544 ms
      ✓ [1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14 544 ms

```

```

  ✓ ApplicationTests (edu.wgu.jobjournalcapstone) 512 ms
    ✓ deleteApplication(Map) 512 ms
      ✓ [1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14 512 ms

```

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
10	Create Unfinished Application	(JSON) Application with a null applied date and a due date.	The application should save to the database, and an ID should be generated.	The application was correctly saved to the database.	PASS
11	Update Application	(JSON) Application with applied date and null due date.	The application information should be updated, and the new data should not match the old data.	The old data does not match the new data.	PASS
7	Delete Application	(JSON) Application with applied date and null due date.	The saved application should be removed from the database, and its ID should not bring up an application.	The application was correctly deleted from the database.	PASS

Final: All tests passed.

```

✓ ApplicationTests (edu.wgu.jobjournalcapstone) 645 ms
  > ✓ createCompleteApplication(Map) 533 ms
  > ✓ createUnfinishedApplication(Map) 25 ms
  > ✓ rejectApplicationWithNoDates(Map) 17 ms
  > ✓ updateApplication(Map) 46 ms
  ✓ deleteApplication(Map) 24 ms
    ✓ [1] jsonPayload={id=-1, dateCreated=2025-05-14T03:07:17.143Z, dateUpdated=2025-05-14T03:07:17.143Z} 24 ms

```

```

✓ UserTests (edu.wgu.jobjournalcapstone) 658 ms
  ✓ duplicateEmailThrowsException(Map) 621 ms
    ✓ [1] jsonPayload={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=Test12345678} 621 ms
  > ✓ findByEmail(String) 13 ms
  ✓ createUser(Map) 24 ms
    ✓ [1] stringObjectMap={id=-1, firstName=Test7132, lastName=User, email=duplicate@email.com, password=Test12345678} 24 ms

```

Section 4. Manual Test Results

The following tests are designed to supplement the JUnit test and focus more on user experience.

4.3.a

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
1	Sign Up	First: "Test" Last: "Account" Email: "test3@test.com" Password: "Password"	The account should be created, and the user should be automatically logged in.	The account is created, but the user is not automatically signed in.	FAIL
2	Create Application - Complete	Job: "Jr Developer" Employer: "Dasani" Posted: 5/13/2025 Applied: 5/14/2025 Status: Awaiting Response	The application should save, and the user should be routed to the application list. The user should be unable to select a posted date after the current date, and the user should only be able to select an applied date between the posted and current date.	The application is saved, and the user is routed to the application list. The UI did not allow me to select dates outside of business requirements.	PASS
3	Create Application - Unfinished	Job: "Sr Developer" Employer: "Google" Posted: 5/13/2025 Due: 5/19/2025 Status: Unfinished	The application should save, and the user should be routed to the application list. The user should be unable to select a posted date after the current date, and the user should only be able to select a due date on or after the current date.	The application is saved, and the date shows as "Due" in the applications list as expected. I was unable to select any unusual dates.	PASS
4	Search - Employer	Input "Dasani" in the search bar.	One option should appear: the Jr. Developer	Search function returned Jr. Developer listing.	PASS

			application at Dasani.		
5	Search - Job	A) Input "Sr" B) Input "Developer"	A) One option should appear – The Senior role at Google. B) Both options should appear.	A) Single relevant option appeared. B) Both options appeared.	PASS
6	Log Out	Press the "Log Out" Button after being logged in.	Log out should direct the user to the home page.	Login did not route the user correctly.	FAIL

4.3.b

ID	Test Name	Input Data	Expected Result	Actual Result	PASS / FAIL
7	Sign Up	First: "Test" Last: "Account" Email: "test3@test.com" Password: "Password"	The account should be created, and the user should be automatically logged in.	The account is now automatically signed in, and the user is taken to the main applications page.	PASS
6	Log Out	Press the "Log Out" Button after being logged in.	Log out should direct the user to the home page.	Log out is now routing the user to the home page.	PASS

Section 6. Summary

Several changes were made during the testing phase. Predominantly in the date logic, as it was where most business logic was concentrated. On the front end, most changes were minor and involved fixing bugs in routing. Further fine-tuning of the error messages for incorrect inputs was also implemented.